



← Back to student guides

ClearML

Experiment tracking

3 levels

116 sections



The Complete Guide

Taught at three levels, written three ways. Pick Beginner, Mid-level, or Senior, then read the full explanation, a fast interview review, or the practical tips and traps for that level.

TASK

QUEUE

AGENT

MODEL

Part three of three. The problems at this level are rarely about syntax. They are about a server thirty people depend on, a GPU bill nobody owns, a model that cannot be rebuilt, and an erasure request against an immutable store. Start with the error table, then the practices, verification, and playbooks underneath it.





Cumulative. Everything from Beginner and Mid still applies. These cause incidents rather than failed runs.

Symptom	Real cause	Fix
A stranger's pull request ran code on your GPUs	Fork PR workflow held workspace credentials	Ephemeral credential-free agents on an isolated queue, or approval-gated
A leaked researcher key read every project	Credentials are workspace-wide by default	One credential per purpose; revoke and rotate on a schedule
A secret was found in a task's Uncommitted Changes	The stored diff captures untracked files	Gitignore discipline, a pre-run check; delete the task and rotate
An agent's cloud credentials were used by someone else's task	Ambient credentials on a shared queue	Per-queue agent pools with per-pool credentials
Experiments stopped reporting metrics, training continued	Elasticsearch crossed the high disk watermark, indices read-only	Free disk, release the block, then fix retention and reporting rate
A restore produced tasks with empty Scalars tabs	Elasticsearch snapshot missing from the backup	All three parts, and a rehearsed restore
A server restart triggered an unplanned migration	Image on latest	Pin the tag; read release notes for every skipped version
Agents broke after a server upgrade	SDK and agent versions unpaired	Upgrade server first; pin an SDK/agent pair across the team
The nightly pipeline silently stopped running	The services agent died; nothing fails loudly	Liveness alert on the services worker
A retention job deleted a production model's weights	No exemption for published models or referenced artifacts	Dry-run for a full cycle; exempt keep , published, and referenced



A model from last year cannot be rebuilt	Base image pinned by tag, not digest; unpinned wheels	Digest + lock file, verified by a weekly rebuild check
A rebuild reproduced but the metric drifted	Unseeded randomness or a CUDA minor change	Connect the seed, record determinism flags, set a tolerance
Nobody could say who approved the production model	The tag was moved by hand in the UI	Promotion as a task, with a threshold and a log
An audit could not name the training data	The run read a raw path instead of a Dataset	Consume by version with <code>alias</code> ; flag runs without one
Cannot honour an erasure request	Personal data lives in the immutable versioned store	Version derived or pseudonymised data; crypto-shred
Deleted objects were unrecoverable	Bucket versioning never enabled	Enable it: highest-priority storage change
Storage tripled with no new datasets	Unpruned sweep artifacts and per-epoch checkpoints	<code>save_top_k_tasks_only</code> , best+last, lifecycle rules
GPU utilisation under 40% while the queue was deep	Abandoned sessions and workers pinned to the wrong queues	Session timeouts; requeue; autoscale to zero
Spot instances cost more than on-demand	No checkpointing, no retries	<code>retry_on_failure</code> plus <code>continue_last_task</code>
A security review failed on access control	Open-source server has one workspace and no RBAC	State it; isolate with queues, credentials, and buckets, or pay
Cost could not be attributed to a team	No prefixes, no mandatory tags	Design attribution in; it cannot be reconstructed later
Two teams' experiments were indistinguishable	Flat project list, ad-hoc naming	A wrapper that sets the prefix and tags



Queues as a security boundary

One agent pool per trust level, each with its own credentials. This is the strongest control the open-source server offers.



One credential per purpose

Per CI pipeline, per agent pool, per service. Revocation becomes surgical and the audit log names a caller.



Workload identity on agents

Bucket access from the node identity. Nothing on disk, nothing to rotate, nothing for a task to steal.



Alert on Elastic disk first

Read-only is silent from the user's side. This one alert prevents the most confusing outage in the system.



Restore, do not just back up

All three parts, rehearsed quarterly, with a published RTO. Mongo alone gives you empty Scalars tabs.



Images by digest

Tags move. A digest plus a lock file is the difference between reproducible and probably reproducible.



Weekly rebuild of a release

The only mechanism that turns reproducibility from a claim into a monitored property.



A wrapper, not a wiki

Twenty lines that make the safe path the shortest path. The only governance that survives a deadline.



01 Prove the fork path is closed

Open a pull request from a fork on a public repo and confirm it cannot enqueue anywhere with credentials. If it can, stop reading and fix that.

02 Find a secret in a diff

Leave an untracked `secrets.txt` in a working tree, run a task, and read it back from Uncommitted Changes. Thirty seconds, and it changes behaviour permanently.

03 Break Elasticsearch on purpose

Fill the volume on a test instance past the high watermark, watch a running task stop reporting, then release the read-only block.

04 Restore and time it

All three backups onto a scratch host. Verify login, scalars, artifact download, agent claim. Publish the number.

05 Kill the services agent

Stop it and see how long before anyone notices. That gap is your detection time, and it is usually days.

06 Rebuild your production model

Clone the task behind it, run it today, compare the metric. Then list every layer it does not pin.

07 Generate a lineage report

From a model id, produce code, data, config, and approval in one command. Note which of the four is missing.

08 Scope an erasure request



09

Measure real utilisation

GPU-hours consumed versus hours that produced a completed run, for one week. Then list the running sessions and their owners.

10

Run the review checklist on someone else's project

It will find something, and the exercise teaches you how much of this is convention rather than enforcement.

The hardening pass every deployment should get

Work down this list. Each step is independently valuable, and the order is by blast radius.

BASH

Copy

```
# 1. Fork trust – the only RCE path in the system
#   Audit every public repo's workflows for ClearML secrets on pull_request.
#   Fork builds → ci-untrusted queue, ephemeral hosts, NO other credentials.

# 2. Bucket versioning – makes every other storage mistake survivable
aws s3api put-bucket-versioning --bucket ml-artifacts \
  --versioning-configuration Status=Enabled
aws s3api put-bucket-versioning --bucket ml-datasets \
  --versioning-configuration Status=Enabled

# 3. TLS at a reverse proxy – no plaintext credential path
#   Terminate at nginx/traefik; never expose 8008/8080/8081 directly.

# 4. Credentials: one per purpose, then delete the shared one
#   Settings → Workspace → App Credentials. Set CLEARML_WORKER_ID per host.

# 5. Artifacts off the file server
#   sdk.development.default_output_uri: "s3://ml-artifacts/<team>/clearml"

# 6. Pin the server image, and record the pair you support
```



- # Elasticsearch volume > 80% → [page](#)
- # MongoDB volume > 80% → [page](#)

lifecycle rules, per prefix

Copy

```
s3://ml-artifacts/<team>/clearml/    30d → IA, 180d → Glacier, 365d → delete
s3://ml-artifacts/<team>/ci/         14d → delete
s3://ml-datasets/                   versioning on; no expiry; write-restricted
```

retention as a scheduled task, dry-run gated

Copy

```
DRY_RUN = os.environ.get("RETENTION_APPLY") != "1"      # safe by default
EXEMPT_TAGS = {"keep", "published", "paper", "release"}
# ...enumerate, print totals, and only delete when explicitly enabled
```

Step	Prevents
Fork isolation	Remote code execution on your GPU fleet
Bucket versioning	Unrecoverable deletion or overwrite
TLS	Credential interception
Per-purpose credentials	A single leak exposing everything
Artifacts in S3	A single-disk failure taking every artifact
Pinned images	An unplanned migration mid-sprint
Disk alerts	The silent metrics outage
Gated retention	Deleting a production model's weights

Do step one today if you have any public repository



queue whose agents hold cloud credentials, treat it as an open door rather than a backlog item.

Verifying, not assuming

Every claim in this section is checkable in a few minutes. Assumed properties are the ones that fail.

Claim	How to verify
"We have backups"	Restore onto a scratch host and open a known task's Scalars tab
"We are reproducible"	Clone last quarter's release task, run it, compare within a tolerance
"Fork PRs are safe"	Open one from a fork and confirm it cannot enqueue
"Credentials are scoped"	Revoke one and observe what breaks
"Artifacts are in S3"	Read an artifact's URL from the UI
"Retention is safe"	Run it in report-only mode and check the exemption logic against a published model
"The registry is authoritative"	Query <code>tags=["production"]</code> and compare against what is serving
"Promotion is recorded"	Find the promotion task's log, user, and threshold
"Elastic has headroom"	<code>_cat/indices</code> sorted by size, plus current disk usage
"The services queue is alive"	Stop it and measure how long detection takes



```
curl -s localhost:8008/debug.ping
curl -s 'localhost:9200/_cluster/health?pretty' | grep -E 'status|unassigned'
curl -s 'localhost:9200/_cat/indices?v&h=index,docs.count,store.size&s=store.size:desc'
df -h /opt/clearml/data/elastic_7 /opt/clearml/data/mongo_4
docker ps --format '{{.Names}}\t{{.Image}}' | grep clearml      # is the tag pinned?
```

the five-minute platform verification, in Python

Copy

```
from clearml import Model, Task

serving = Model.query_models(project_name="vision", model_name="resnet18-cls",
                             tags=["production"], max_results=5)
print("production models:", [(m.id, m.published) for m in serving])
assert len(serving) == 1, "exactly one production model expected"
assert serving[0].published, "production model is not published – it can be swapped"

t = Task.get_task(task_id=serving[0].task)
assert not t.data.script.diff, "production model was trained from a dirty tree"
assert t.data.container, "no container recorded – the environment is not pinned"
print("commit:", t.data.script.version_num, "image:", t.data.container)
```

Turn the assertions into a scheduled task

Those six lines are a continuously-verified statement about your production model: exactly one, published, trained from a clean tree, in a recorded container. Run them weekly on the services queue and you find out when someone breaks one of those properties, rather than during an audit.

Cost governance

Cost becomes your problem the moment ClearML is shared, and it is entirely a measurement problem before it is a technical one.



what did we spend on GPUs?	Runtime summed per queue, from task timestamps
How much of that produced nothing?	Runtime of failed and aborted tasks
Who is holding idle hardware?	Running clearml-session tasks and their owners
What is filling the bucket?	Size by prefix, then by tag within a prefix
What fraction is sweep waste?	Artifacts on non-top-K HPO trials
Which team spent it?	Mandatory team: and cost-centre: tags

ops/cost_report.py: weekly, on services

Copy

```
from datetime import datetime, timedelta
from collections import defaultdict
from clearml import Task

week = datetime.utcnow() - timedelta(days=7)
by_queue, wasted, by_team = defaultdict(float), defaultdict(float), defaultdict(float)

for t in Task.get_tasks():
    if not t.data.completed or t.data.completed < week:
        continue
    runtime = (t.data.completed - t.data.started).total_seconds() / 3600
    queue = t.data.execution.queue or "unknown"
    by_queue[queue] += runtime
    if t.status in ("failed", "aborted"):
        wasted[queue] += runtime
    team = next((x.split(":", 1)[1] for x in t.get_tags() if x.startswith("team:")), "unknown")
    by_team[team] += runtime

logger = Task.current_task().get_logger()
for q, hours in sorted(by_queue.items(), key=lambda kv: -kv[1]):
    logger.report_single_value(f"hours_{q}", round(hours, 1))
    logger.report_single_value(f"wasted_{q}", round(wasted[q], 1))
for team, hours in by_team.items():
    logger.report_single_value(f"hours_team_{team}", round(hours, 1))
```



Session timeouts	Often the largest single win	Occasional annoyance for a long debugging session
Fail-fast validation	Whole runs that would have died at hour six	Ten lines at the top of the script
Verified HPO objective + iteration caps	Entire sweeps	Two minutes of checking
<code>save_top_k_tasks_only</code>	Most sweep storage	You lose the non-top trials' artifacts
Lifecycle rules	Steady, compounding	Old artifacts become slow or gone to retrieve
Autoscale to zero	Idle-hour elimination	Cold-start latency on the first job
Spot with retries	60–80% on interruptible work	Requires checkpointing and resumable training

Do not tier storage without pricing retrieval

Glacier looks cheap until someone needs forty checkpoints back for an audit, and archive retrieval charges plus egress can exceed a year of standard storage. Model the read pattern, not just the shelf price, and keep anything a rebuild check or an audit might need in a warm class.

Incident playbooks

Four incidents, each with a first action, a verification, and the prevention. Keep these in the repository.

The server is gone.

01

Do not rush the version



02 **Mongo first, then Elasticsearch**

Mongo is the system of record; Elastic restores metrics and logs.

03 **Verify in order**

Login → a known task's scalars → an artifact download → an agent claiming a queued task.

04 **Reconcile**

Tasks that finished during the outage may show as running. Mark them and record the gap.

05 **Publish the actual RTO**

Compare it against your rehearsed number and fix whichever step was slower than expected.

Elasticsearch went read-only. Symptom: metrics stop, training continues. Free disk (delete old indices or expand the volume), release the block with a `_settings` PUT, then fix the cause: reporting rate, retention, or capacity. Prevention: an 80% disk alert.

A model cannot be rebuilt. Run the lineage report and find the unpinned layer; it is nearly always the base image tag or an unpinned wheel. If artifacts are gone, check whether retention deleted the task, then check bucket versioning. Record what could not be recovered. That list is your next sprint.

Unauthorised enqueue. Treat it as a credential compromise, not a mistake: abort the task, revoke the credential that enqueued it, audit everything that queue's agents could reach with their ambient credentials, rotate anything exposed, then fix the queue isolation that permitted it.

Incident	First action	Prevention
Server lost	The same pinned image	Tested three-part backups; a published RTO
Elastic read-only	Free disk, release the block	80% disk alert; retention; per-epoch reporting



Model unbuildable	Lineage report	Digests, lock files, weekly rebuild check
Unauthorised enqueue	Abort, revoke, rotate	Queue isolation; scoped credentials; no fork access
Nightly silently stopped	Check the services agent	Liveness alert on the services worker
Storage spike	Break down by prefix and tag	Lifecycle rules; <code>save_top_k_tasks_only</code> ; retention
GPU queue full of nothing	List sessions, name owners	Session timeouts; weekly utilisation report

A playbook is only real once someone else has followed it

Have a person who did not build the platform work through one, and write down where they got stuck. Those gaps (an unstated hostname, an assumed credential, a missing sudo) are exactly the ones that matter at 2am.

Running ClearML as a platform

Five obligations. This is what "platform ownership" means day to day.

Obligation	Concretely
Publish the numbers	Weekly: GPU utilisation, queue depth, storage by team, rebuild-check result
Ship a wrapper	Project prefix, <code>output_uri</code> , tags, image digest, package pinning: as defaults
Own the upgrade cadence	A tested quarterly window with a rollback, announced in advance



Alert on the platform	Elastic disk, services liveness, queue depth, mongo disk, API 5xx
Keep four runbooks	Written, in the repo, and validated by an outside reader

The alert list, in priority order, and it is deliberately short:

01 Elasticsearch disk > 80%

Read-only stops all metric reporting silently. Highest-value alert in the system.

02 Services agent liveness

No heartbeat for ten minutes. Pipelines, schedulers, and triggers all stop with no error anywhere.

03 Queue depth sustained over an hour

A dead worker, a stuck session, or a runaway sweep.

04 MongoDB disk > 80%

The system of record.

05 API server 5xx rate

Agents fail to claim; users experience it as hangs.

06 Rebuild check failing

Reproducibility has regressed and nothing else would have told you.

07 Weekly GPU utilisation < 40%

You are paying for idle hardware.

Task failures do not belong on the platform alert list

A failed training run is its author's problem and they already have the link. Paging the platform team on task failures trains everyone to ignore the channel, and the alerts that



Machine-learning specifics

A few things are ClearML-and-ML rather than general platform work, and they come up in senior interviews.

Checkpoint policy is a cost decision, not a training decision. Best plus last, not every epoch. Every-epoch checkpointing of a 2 GB model over a 100-epoch run is 200 GB per experiment, and nobody ever loads epoch 47.

Sweeps need a storage plan before they need a search strategy. Forty trials × 2 GB is 80 GB per sweep. `save_top_k_tasks_only` is the difference between a weekly sweep and a quarterly storage escalation.

Determinism is a spectrum, and you should record where you sit. Seeding gets you most of the way; `torch.use_deterministic_algorithms(True)` costs speed; some kernels are non-deterministic regardless. Record the flags and set a metric tolerance in the rebuild check rather than pretending bit-exactness.

Debug samples are data. Images and sample outputs land in Elasticsearch and object storage, and if the inputs are personal data those samples are copies nobody remembers during an erasure request. Report them sparsely and include them in your deletion scope.

Model weights are the un-erasable artefact. For some model classes, training data is extractable. Which is why the erasure conversation is architectural, keep personal data out of the versioned store, rather than a runbook.

A published model is the only thing that cannot be silently swapped. An unpublished "production" model can have its weights replaced by anyone with workspace access, and serving would pick up the change with no record. Publishing is a cheap, real control.

The one-sentence version of this whole page



Everything else here is an instance of those two.

The checklist to run before shipping

Area	Check
Fork trust	Can a fork PR enqueue? Do those agents hold cloud credentials?
Credentials	One per purpose? Nothing workspace-wide in automation? Rotation rehearsed?
Queue isolation	Are queues a boundary, with distinct credentials per pool?
Diffs	Has anyone audited recent tasks' Uncommitted Changes for secrets?
TLS	Server behind a proxy, no plaintext credential path?
Storage	Artifacts in S3, not the file server? Bucket versioning on? Lifecycle per prefix?
Backups	All three parts? Restored in the last quarter? RTO published?
Pinning	Server image tag pinned? SDK/agent pair recorded? Base images by digest ?
Elastic	80% disk alert? Retention scheduled? Reporting per epoch?
Retention	Dry-run gated, exempting published, keep -tagged, and referenced artifacts?
Reproducibility	Lock file, seeds connected, determinism recorded, weekly rebuild check green?
Lineage	Code, data, config, and approval for the production model in one command?
Promotion	A task with a threshold and a log? Winner published? Exactly one production tag?
Serving	Behind auth? Following a tag, not an id? Canary path available?
Cost	Utilisation measured and published? Session timeouts? save_top_k_tasks_only ?



Services queue	Liveness alert? treated as production?
Runbooks	Four written, and followed by someone who did not write them?
Boundaries	Documented where ClearML stops and what owns the rest?

Nobody passes this on the first attempt. Two failures matter disproportionately: **an untested restore** and **a fork pull request with credentials**. One is unrecoverable data loss, the other is remote code execution. Rank by blast radius rather than by effort, fix those two first, and publish the exceptions you are choosing to live with along with the reason. A checklist with argued exceptions is a better artefact than a green one, because it shows the trade-offs were made deliberately and it gives whoever inherits the platform a map instead of a mystery.



PREVIOUS

Senior · Interview Review



Free MLOps and AI learning for engineers across the Middle East and North Africa — live sessions, three structured roadmaps, open cohort courses, study groups, and mentorship. We also train, staff, and build for companies across the region.





Courses

The MLOps Practitioner

Sessions

Articles

COMMUNITY

Meet the team

Services for companies

FAQ

Contact us

Privacy Policy

WHERE WE ARE

WhatsApp

Discord

LinkedIn

X

YouTube

GitHub

Zomra



Brainsmingle space

Free to join this week with code **FREECORETEAM**